

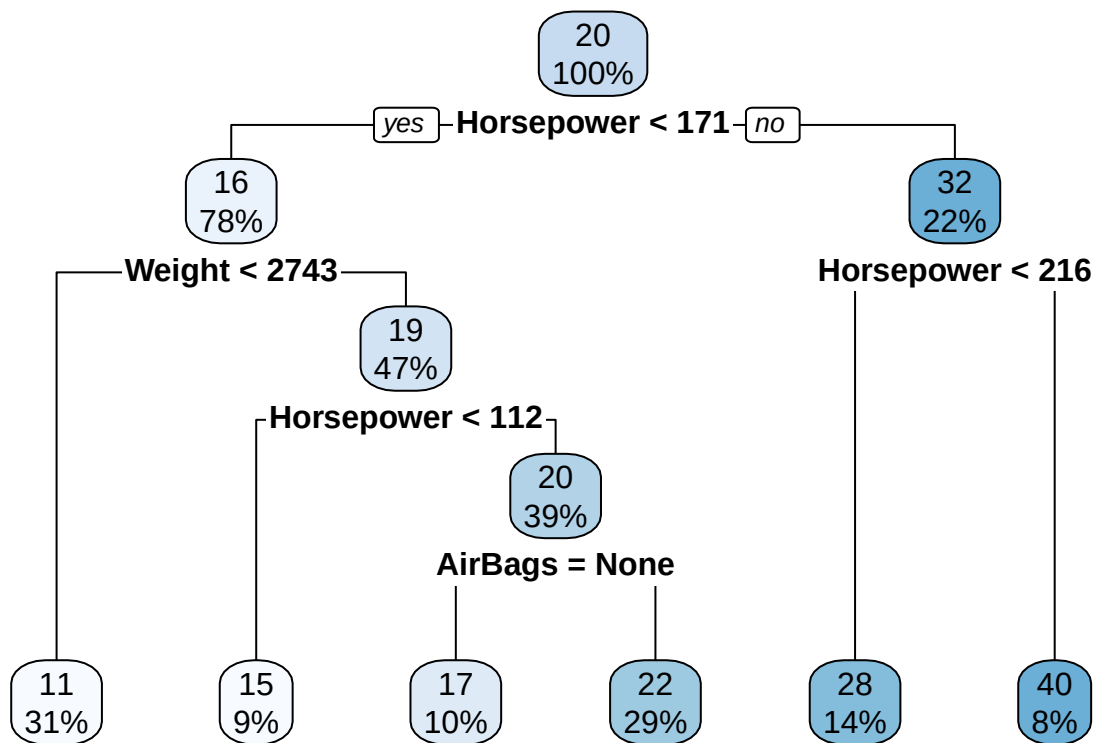
Homework 9

Moshood Abiola

2025-09-25

(1.) This problem will step you through fitting a regression tree model and making a prediction

(a.) Use the Cars93 dataset in the MASS library. Create a tree for predicting Price from Horsepower, Weight, Length, whether a car has AirBags, and the Origin of production.



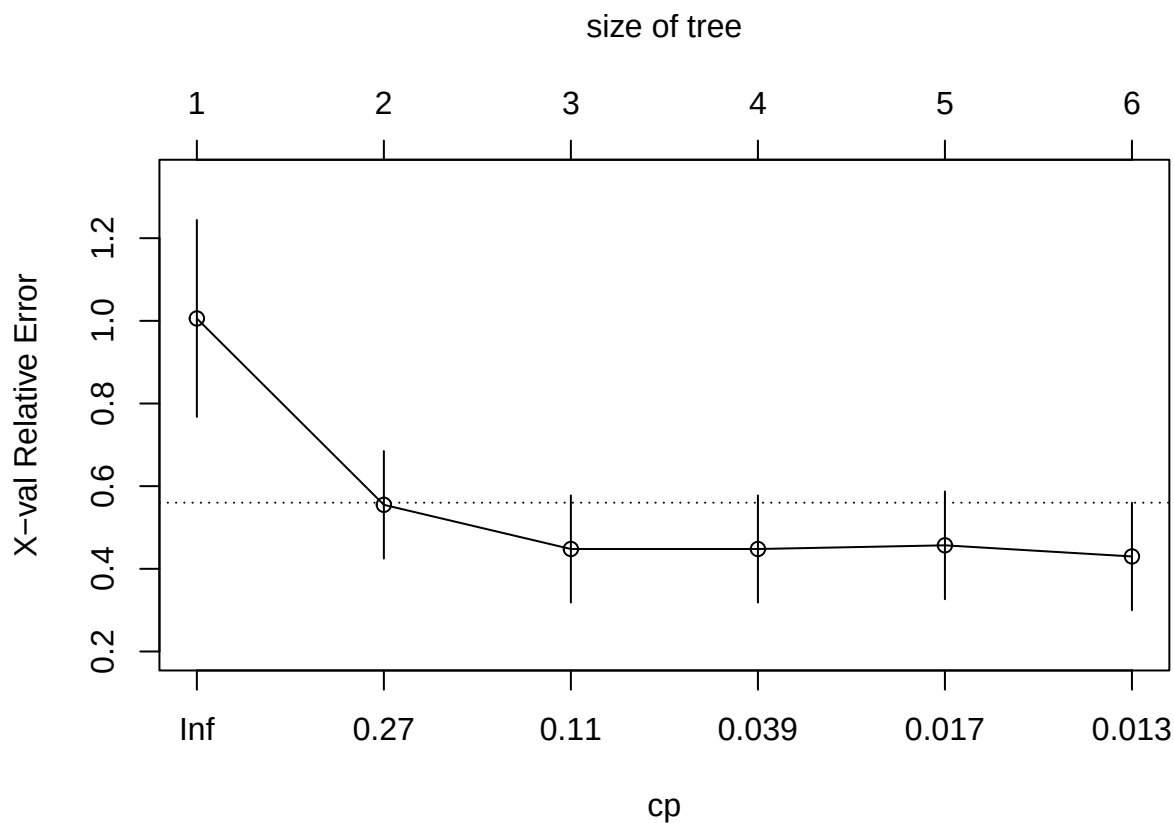
(b.) Predict the price of a car with 200 horsepower, weighs 2705 pounds, is 177 inches long, has

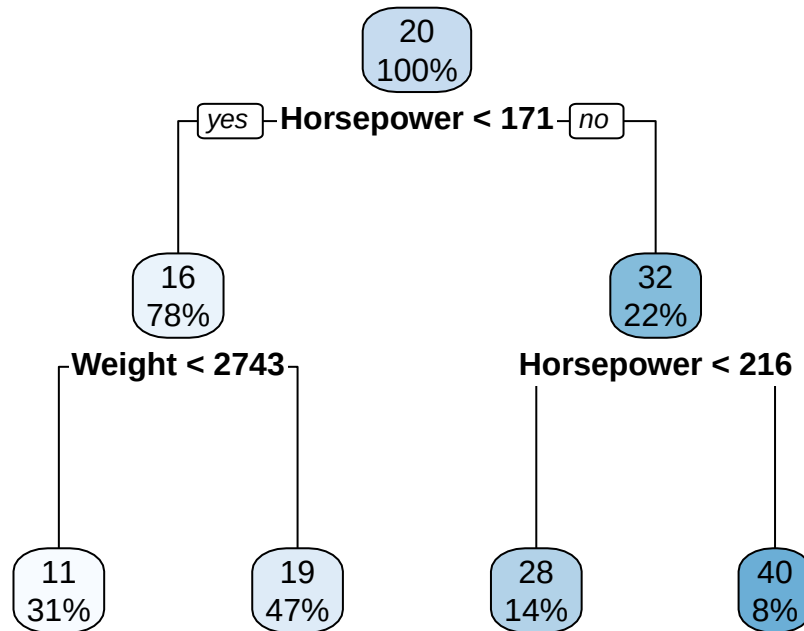
driver and passenger airbags, and was not made in the US.

1
27.79231

(c.) Use `plotcp()` to decide on an optimal `cp` value. You have a little flexibility in your choice, but explain why you picked the one you did. Use `prune()` to apply the chosen `cp`, and plot the pruned tree.

```
##
## Regression tree:
## rpart(formula = Price ~ Horsepower + Weight + Length + AirBags +
##       Origin, data = Cars93, method = "anova")
##
## Variables actually used in tree construction:
## [1] AirBags    Horsepower Weight
##
## Root node error: 8584/93 = 92.301
##
## n= 93
##
##      CP nsplit rel error  xerror  xstd
## 1 0.473134     0  1.00000 1.00577 0.23823
## 2 0.152304     1  0.52687 0.55488 0.13002
## 3 0.081602     2  0.37456 0.44793 0.12961
## 4 0.018329     3  0.29296 0.44793 0.12961
## 5 0.016055     4  0.27463 0.45687 0.13038
## 6 0.010000     5  0.25858 0.42992 0.13010
```





2. (Grad student only) I wrote most of a function to implement Algorithm 9.2, but left the innermost parts out. I also provided two functions to aid the construction and retrieval of all subsets of levels of a categorical variable.

(a.) Complete the function

```

# I fix the first level on the LEFT to avoid duplicates like {A}|{B,C} and {B,C}|{A}.
level_partitions <- function(levels_vec) {
  L <- sort(unique(levels_vec)); k <- length(L)
  if (k <= 1) return(list())
  first <- L[1]; others <- L[-1]
  out <- list()
  # iterate all subsets of 'others'; exclude empty (only {first}) and full (all levels)
  for (m in 1:(2^(k-1) - 1)) {
    pick <- as.logical(intToBits(m))[seq_along(others)]
    left <- c(first, others[pick])
    if (length(left) < k) out[[length(out) + 1]] <- left
  }
  out
}

# Weighted mean and weighted SSE
wmean <- function(x, w) sum(w * x) / sum(w)

```

```

wsse <- function(x, w) { mu <- wmean(x, w); sum(w * (x - mu)^2) }

best_split_alg9_2 <- function(X, y,
                             weights = NULL,
                             minsplit = 2,      # small defaults for classroom toy data
                             minbucket = NULL,
                             cp = 0) {

  n <- length(y)
  if (is.null(weights)) weights <- rep(1, n)
  if (n < minsplit) return(NULL)
  if (is.null(minbucket)) minbucket <- max(1, floor(minsplit / 3))

  # 1) baseline goodness = SSE of parent node
  sse_parent <- wsse(y, weights)

  # initialize
  best <- list(goodness = Inf, split_var = NA, split_val = NA,
              left_idx = integer(0), right_idx = integer(0),
              improvement = 0, type = NA)

  # 2) loop over columns of X
  for (j in seq_len(ncol(X))) {
    xj <- X[[j]]

    # (a) numeric
    if (is.numeric(xj)) {
      ux <- sort(unique(xj))
      if (length(ux) > 1) {
        mids <- (ux[-1] + ux[-length(ux)]) / 2
        for (thr in mids) {
          L <- which(xj <= thr); R <- which(xj > thr)
          if (length(L) < minbucket || length(R) < minbucket) next
          sse_LR <- wsse(y[L], weights[L]) + wsse(y[R], weights[R])
          improve <- sse_parent - sse_LR
          if (sse_LR < best$goodness && improve >= cp * sse_parent) {
            best <- list(goodness = sse_LR, split_var = names(X)[j],
                        split_val = thr, left_idx = L, right_idx = R,
                        improvement = improve, type = "numeric")
          }
        }
      }
    }

    # (b) categorical
  } else {
    levs <- unique(xj)
    if (length(levs) > 1) {
      parts <- level_partitions(levs)
      for (left_levels in parts) {
        L <- which(xj %in% left_levels); R <- setdiff(seq_len(n), L)
        if (length(L) < minbucket || length(R) < minbucket) next
        sse_LR <- wsse(y[L], weights[L]) + wsse(y[R], weights[R])
        improve <- sse_parent - sse_LR
      }
    }
  }
}

```

```

        if (sse_LR < best$goodness && improve >= cp * sse_parent) {
          best <- list(goodness = sse_LR, split_var = names(X)[j],
                      split_val = left_levels, left_idx = L, right_idx = R,
                      improvement = improve, type = "categorical")
        }
      }
    }
  }

  if (is.infinite(best$goodness)) return(NULL)
  best
}

```

2.b Test the completed function on the toy dataset in Example 9.1 and ensure it returns the same

answer we obtained by hand

```

## split_var : X_1

## split_val : 7

## goodness   : 6.75

## left idx   : 1 3 4 5

## right idx  : 2

```